SEAT BOOKING SYSTEM

VITyarthi – Build Your Own Project

Python-based student project

1. Introduction

This project is a simple seat booking program written completely in Python. A user can see the available seats, book a seat, cancel a booking, search for a booking and view all current bookings.

2. Problem Statement

Managing seats manually can become confusing, especially when more than one person wants the same seat. The program keeps track of booked seats and reduces the chance of duplicate bookings.

3. Objectives

The main goals are to display seats clearly, prevent duplicate bookings, save booking details, allow cancellation and search, and demonstrate Python concepts such as classes, file handling, validation and testing.

4. Functional Requirements

The system provides seat management, booking, cancellation, search, booking-list and storage functions. It has clear inputs and outputs and a menu-based workflow.

5. Non-functional Requirements

The program focuses on usability, quick local operations, reliable file storage, maintainable code, basic error handling and low resource use.

6. System Architecture

The user works through the console menu. BookingService contains the main rules and works with SeatManager for seats and BookingStore for saved data.

7. Design Diagrams

Workflow, use-case, class, sequence, architecture and storage diagrams are included in the docs folder.

8. Design Decisions and Rationale

The program uses small classes so each part has one main responsibility. JSON was chosen because the project is small. A console interface keeps the Python logic easy to see and explain.

9. Implementation Details

The implementation uses dataclasses, JSON, pathlib, datetime and unittest. Before saving a booking, the program checks the name, show, seat number and whether the seat is already booked.

10. Screenshots / Results

The project was run with sample inputs. The screenshots show the seat map, booking process, duplicate-booking handling and automated test results.

11. Testing Approach

Four automated tests check booking, duplicate-seat prevention, invalid seats and cancellation. The menu and other user interactions were also checked manually.

12. Challenges Faced

The main challenges were keeping the seat status correct, preventing duplicate reservations, saving data properly and keeping the code simple.

13. Learnings and Key Takeaways

The project demonstrates Python classes, functions, file handling, JSON, input validation, testing and basic software design.

14. Future Enhancements

Possible future additions include multiple shows, different seat types, a GUI, database storage, login accounts and online booking.

15. References

VITyarthi Build Your Own Project – General Project Instructions & Submission Guidelines. Python standard-library documentation for the modules used.

Program Demo

```
Seat Booking System – Humanized Demo Run


Welcome to the Seat Booking System!
You can manage your event seats from the menu below.

========== SEAT BOOKING SYSTEM ==========
1. View seat map
2. Book a seat
3. Cancel a booking
4. Search for a booking
5. View all bookings
6. Exit
========================================
Enter your choice (1-6):
========== SEAT MAP ==========
[A1] [A2] [A3] [A4] [A5] [A6]
[B1] [B2] [B3] [B4] [B5] [B6]
[C1] [C2] [C3] [C4] [C5] [C6]
[D1] [D2] [D3] [D4] [D5] [D6]
[E1] [E2] [E3] [E4] [E5] [E6]

X = Already booked


========== SEAT BOOKING SYSTEM ==========
1. View seat map
2. Book a seat
3. Cancel a booking
4. Search for a booking
5. View all bookings
6. Exit
========================================
Enter your choice (1-6):
========== SEAT MAP ==========
[A1] [A2] [A3] [A4] [A5] [A6]
[B1] [B2] [B3] [B4] [B5] [B6]
[C1] [C2] [C3] [C4] [C5] [C6]
[D1] [D2] [D3] [D4] [D5] [D6]
[E1] [E2] [E3] [E4] [E5] [E6]

X = Already booked

Let's book your seat.
Enter customer name: Enter seat number (example: A1): Enter show/event name:
Great! Your seat has been booked.

---------- BOOKING DETAILS ----------
Booking ID : B20260929170621076317
Name       : Aman
...
[output shortened]
```

Automated Test Results

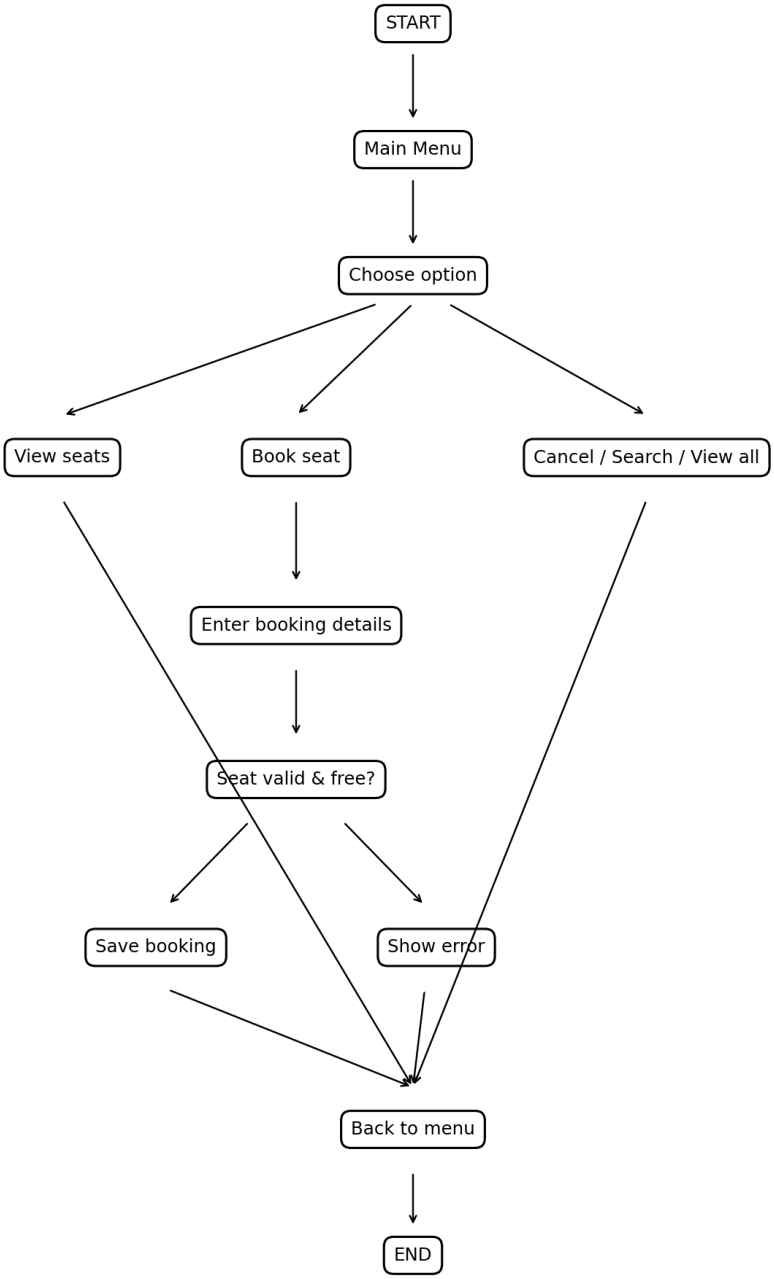
```
Seat Booking System — Humanized Test Results

Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_on_startup
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 720, in _warm_feature_flows
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 704, in _warm_collaboration_flows
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/generated/interface/models.py", line 32317, in hydrate_crdt_from_proto
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/remote.py", line 749, in __call__
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/client.py", line 150, in call
artifact_tool.rpc.client.RemoteError: hydrateCrdtFromProto requires an empty collaborative document.
test_booking_can_be_cancelled (test_seat_booking.TestSeatBookingSystem.test_booking_can_be_cancelled) ... ok
test_free_seat_can_be_booked (test_seat_booking.TestSeatBookingSystem.test_free_seat_can_be_booked) ... ok
test_invalid_seat_is_rejected (test_seat_booking.TestSeatBookingSystem.test_invalid_seat_is_rejected) ... ok
test_same_seat_cannot_be_booked_twice (test_seat_booking.TestSeatBookingSystem.test_same_seat_cannot_be_booked_twice) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.003s

OK
```

Flowchart


START
Main Menu
Choose option
View seats
Book seat
Cancel / Search / View all
Enter booking details
Seat valid & free?
Save booking
Show error
Back to menu
END